

scitech-librarian — Manuel utilisateur

version 3.4.2

2026-09-04

Table des matières

1. Ce que c'est	2
2. Installation et configuration	2
3. Concepts	3
4. Écrire des requêtes	3
5. Lancer une recherche	4
6. Le répertoire de recherche	5
6.1 Index	5
6.2 Apporter des notices de l'extérieur	5
6.3 Depuis Zotero, Mendeley et EndNote	6
7. Rapports	6
7.1 Une exécution	6
7.2 Le répertoire de recherche entier	6
7.3 Niveaux	6
7.4 Formats	7
7.5 Filtres	7
7.6 PRISMA	7
7.7 Suggestions	8
7.8 Langues	8
8. Indicateurs des revues	8
9. Web of Science	9
10. Journaux et audit	9
11. Flux de travail	9
12. Fonctionnalités et limitations	10
13. Tests	10
14. Licence et conduite	11

[English](#) · [Português \(Brasil\)](#) · [Español](#) · [Deutsch](#) · **Français**

Traduction du manuel anglais, qui fait référence ; les commandes, noms de fichiers, options et blocs de code sont conservés tels quels.

1. Ce que c'est

scitech-librarian est un instrument de recherche bibliographique reproductible pour les sciences et l'ingénierie. Vous écrivez une requête structurée une seule fois ; il l'exécute contre jusqu'à huit bases de données bibliographiques via leurs API documentées, archive tout (notices, la chaîne de requête exacte envoyée à chaque base de données, nombres de résultats, un journal) et rédige un rapport de recherche bibliographique avec un diagramme de flux PRISMA 2020. Au fil des mois, les exécutions, plus les notices obtenues par d'autres moyens, s'accumulent dans un **répertoire de recherche** que le même rapport peut décrire dans son ensemble — ce que chaque recherche a apporté, ce que chaque base de données a contribué, comment les nombres ont dérivé, quelles revues comptent.

Ce sont cinq scripts Python plus deux modules partagés (**render.py**, **i18n.py**), sans dépendance au-delà de la bibliothèque standard. Il n'y a rien à installer : copiez les fichiers, fournissez vos clés, écrivez **queries.json**, exécutez.

Fichier	Rôle
librarian.py	exécute une recherche ; archive une exécution ; appelle le rapport
project.py	répertoire de recherche : index, ingestion de notices externes, état
report.py	rapports pour une exécution ou le répertoire entier ; PRISMA ; filtres
journals.py	indicateurs des revues (chiffres de type facteur d'impact) par année
wos_manual.py	Web of Science à la main (pas d'API gratuite utilisable)
render.py	moteurs de rendu Markdown / HTML / LaTeX / texte et la chaîne PDF (importé par report.py)
i18n.py	langues du rapport : le catalogue en / pt-BR / es / de / fr (importé par report.py ; §7.8)

Pour les agents d'IA. **AGENTS.md** à la racine du dépôt est la description complète de l'outil orientée machine. Si vous travaillez avec un agent de programmation (Claude Code, Codex, Cursor...), dites-lui : « *Lis AGENTS.md, puis fais une vérification de nouveauté sur X* » — il contient les commandes, les schémas de fichiers, les flux de travail et les règles que l'agent ne doit pas enfreindre.

2. Installation et configuration

Prérequis : Python 3.9 ou plus récent. Facultatif, pour des rapports PDF composés : une distribution LaTeX (xelatex, lualatex ou pdflatex) ou pandoc ; sans eux, le PDF est produit par un moteur texte brut intégré.

```
git clone https://github.com/fabiocampolim-design/scitech-librarian
cd scitech-librarian
cp .env.example .env # fill in what you have (or use the environment)
cp queries.example.json queries.json
python librarian.py --selftest
```

Les clés sont lues depuis l'environnement du processus. Copiez **.env.example** vers **.env** et remplissez ce fichier, ou définissez les mêmes noms de variable dans votre shell, dans la configuration de votre agent ou lanceur, ou comme secrets de CI — **.env** ne fournit que ce que l'environnement n'a pas déjà défini, donc une variable définie à l'extérieur l'emporte toujours et le fichier est facultatif.

Clés :

Clé	Nécessaire pour	Comment l'obtenir
CONTACT_EMAIL	accès « polite pool » à OpenAlex/Crossref/Unpaywall	votre adresse

Clé	Nécessaire pour	Comment l'obtenir
ADS_TOKEN	NASA ADS	gratuit, https://ui.adsabs.harvard.edu/user/settings/token
SCOPUS_API_KEY	Scopus (+ réseau institutionnel/VPN)	gratuite, https://dev.elsevier.com/apikey/manage
SCOPUS_INSTTOKEN	Scopus sans VPN	demandez à votre bibliothèque
S2_API_KEY	Semantic Scholar plus rapide	facultatif
CORE_API_KEY	CORE (si configuré dans backends.json)	gratuite, https://core.ac.uk/services/api
WOS_STARTER_KEY	Web of Science Starter API (grammaire restreinte)	rarement rentable

Cinq backends (OpenAlex, arXiv, INSPIRE-HEP, Semantic Scholar, Crossref) n'ont besoin ni de clé ni d'institution. `python librarian.py --list` indique quelles clés ont été trouvées par l'une ou l'autre voie ; `--selftest` prouve qu'elles fonctionnent.

Utilisation intégrée dans un autre projet. Placez les sept fichiers dans un sous-répertoire `tools/` ; `.env`, `queries.json` et `lit/` sont alors cherchés dans le répertoire parent.

3. Concepts

Bloc. Une requête structurée : une liste de groupes de synonymes combinés par AND, chaque groupe étant une liste de synonymes combinés par OR. Un bloc a un nom (A, CD, NOV...), un titre et une note. Les blocs vivent dans `queries.json`.

Exécution. Un lancement de `librarian.py` : chaque bloc sélectionné contre chaque backend sélectionné, archivé sous `lit/runs/<timestamp>/`.

Répertoire de recherche. Un dossier (par défaut `lit/`, un autre avec `--outdir`) contenant toutes les exécutions d'un projet, les notices ingérées de l'extérieur, l'index du projet (`project.json`), les nombres du tri PRISMA (`screening.json`), les indicateurs des revues, les rapports et les journaux d'audit. Un répertoire par projet ; un laboratoire en a plusieurs.

Source manuelle. Des notices qui ne viennent pas d'une exécution : un export Zotero ou Mendeley, le fichier RIS d'un collègue, une session Web of Science, une liste de références. Ingérées avec `project.py ingest`, elles gardent leur provenance (qui, quand, d'où, méthode) et apparaissent dans chaque rapport comme une source de plus, et dans le flux PRISMA dans la colonne appropriée.

Notice. Le schéma commun utilisé par chaque fichier : `title year doi journal authors url abstract cited_by issn block backend`. Les notices de projet fusionnées portent aussi `found_by` (les sources qui l'ont trouvée) et `first_seen`.

Niveau. Ce qu'un rapport contient : `simple` (quelques pages), `intermediate` (chaque notice unique plus des analyses), `full` (tout, résumés compris — des centaines de pages pour les grands projets).

4. Écrire des requêtes

`queries.json` :

```
{
  "PSC": {
    "title": "Perovskite solar cell degradation under humidity",
    "note": "grounding block -- expect thousands",
    "groups": [["perovskite solar cell", "halide perovskite photovoltaic"],
               ["degradation", "stability"],
               ["humidity", "moisture"]]
```

```

},
"NOV": {
  "title": "novelty check: origami metamaterials as topological acoustic pumps",
  "note": "a SMALL number is the good outcome; read every hit",
  "groups": [["origami", "kirigami"], ["acoustic metamaterial", "phononic crystal"],
    ["topological pumping", "edge state"]],
  "arxiv_groups": [0, 2]
}
}

```

Règles pratiques :

- Ne mettez pas les termes entre guillemets; l'outil le fait selon la grammaire de chaque base de données.
- Un mot générique seul (**model**, **structure**, **system**) dans son propre groupe est la cause habituelle de nombres dans les dizaines de milliers.
- **arxiv_groups** indique quels groupes (deux au plus) arXiv reçoit; arXiv se bloque sur les booléens profondément imbriqués. Par défaut : les deux premiers. arXiv est paginé par 100 notices avec une pause de 3 s, donc un grand `--limit` y est lent.
- Le bloc le plus informatif est une intersection délibérée de deux littératures dont vous soupçonnez qu'elles ne se parlent pas. Un résultat proche de zéro est une découverte — *si* vous lisez ensuite chaque résultat.
- Les opérateurs de proximité (**NEAR/n**, **W/n**) ne sont pas exprimables; si votre article en a besoin, gardez à côté des chaînes Web of Science / Scopus écrites à la main et citez celles-ci.

5. Lancer une recherche

```

python librarian.py                # all blocks, every configured backend
python librarian.py --counts-only  # hit counts only (seconds)
python librarian.py --blocks NOV CD # selected blocks
python librarian.py --backends openalex ads
python librarian.py --skip arxiv   # drop a misbehaving backend
python librarian.py --limit 500    # records per block and backend (default 300)
python librarian.py --pdfs --pdf-blocks NOV # legal OA-PDF links via Unpaywall
python librarian.py --keep-junk    # keep non-curated venues (Zenodo, SSRN...)
python librarian.py --outdir lit_topomat # another research directory
python librarian.py --report-level intermediate --report-format md html pdf
python librarian.py --report-lang pt-BR # report in Portuguese (en, pt-BR, es, de, fr; §7.8)
python librarian.py --no-report
python librarian.py --queries other.json # another query file (default ./queries.json)
python librarian.py --backends-file b.json # another backends config; --init-backends writes the default
python librarian.py --timeout 60          # per-request timeout in seconds (default 45)
python librarian.py --list                # blocks and backend readiness, then exit
python librarian.py --selftest            # ping every backend, then exit

```

Liste complète des paramètres : `python librarian.py --help`. Chaque option a une valeur par défaut; `--outdir`, `--verbose`, `--quiet` et `--log-dir` existent sur chaque script.

Ce qu'une exécution écrit (`lit/runs/<stamp>/`) :

Fichier	Contenu
<code>counts.json</code> , <code>counts.md</code>	nombres de résultats par bloc et backend; tableau prêt à coller
<code>queries.json</code>	la chaîne de requête exacte envoyée à chaque backend
<code>blocks.json</code>	les définitions de blocs utilisées
<code>meta.json</code>	réglages, backends et points d'accès, version, durées
<code>records/<block>_<backend>.json</code>	notices brutes par backend (après le filtre de revues)
<code>ris/<block>_<backend>.ris</code>	RIS par bloc pour Zotero/Mendeley/EndNote

Fichier	Contenu
all_records.json/.csv/.ris	dédoublonnées, triées par citations
all_records.bib, all_records.csl.json	le même ensemble en BibTeX et CSL-JSON
junk.json	notices retirées par le filtre de revues, avec leurs revues
prisma.json	modèle pour les étapes PRISMA manuelles
run.log	tout ce qui a été affiché
report.*	le rapport (voir §7)

Plus `lit/counts_history.csv` (une ligne par bloc/backend/exécution, pour la dérive) et `lit/logs/librarian_<stamp>_<pid>`. (journal d'audit : invocation, versions, chaque message).

Les nombres sont sauvegardés après chaque appel d'API et Ctrl-C est sûr : un blocage tardif dans une longue exécution ne perd rien.

6. Le répertoire de recherche

6.1 Index

```
python project.py init --name "Topological materials review" --description "..."  
python project.py status
```

`status` liste chaque membre (exécutions et sources manuelles) avec date, nombre de notices, méthode et étiquette, l'état du dossier de dépôt et le dernier rapport. Les membres sont découverts en listant le répertoire — rien n'a besoin d'être déclaré. `project.json` ne contient que ce qui ne peut pas être découvert :

```
{"name": "...", "description": "...", "created": "2026-08-28",  
  "exclude": ["20260814T223331"],  
  "labels": {"20260828T095041": "August full scan"},  
  "block_aliases": {"X": "CD"},  
  "defaults": {"level": "simple", "format": ["md"], "metric": "openalex_2yr"}}
```

```
python project.py exclude 20260814T223331      # a test run you do not want in reports  
python project.py label 20260828T095041 "August full scan"  
python project.py alias X CD                    # block renamed between runs  
python project.py oa                            # Unpaywall pass over every member that lacks OA data  
python project.py oa --members 20260828T095041 # restrict the pass to these member ids
```

`oa` est la recherche de libre accès a posteriori : les exécutions faites sans `--pdfs` et les sources manuelles reçoivent les champs `is_oa` / `oa_pdf` (copies légales uniquement, mises en cache dans `unpaywall_cache.json`), que les statistiques de libre accès du rapport et `--oa-only` couvrent alors pour tout le projet.

6.2 Apporter des notices de l'extérieur

Trois façons, toutes aboutissant dans `lit/manual/<name>/` avec le fichier d'origine, un `records.json` au schéma commun et un `source.json` avec la provenance :

1. **Ligne de commande** — la façon entièrement décrite :

```
python project.py ingest export.ris --name zotero-aug --block CD \  
  --method citation --who "A. Colleague" --origin "Zotero group library" \  
  --note "reference lists of the three key papers"
```

Plusieurs fichiers peuvent être donnés ; `--kind` remplace la détection par extension (`ris`, `bibtex`, `csv`, `json`).

2. **Dossier de dépôt** — déposez des fichiers dans `lit/inbox/` et lancez `python project.py ingest --inbox` ; chaque fichier devient une source nommée d'après lui (ajoutez `--method` etc. pour l'appliquer à tous).

3. **Web of Science** — `python wos_manual.py ingest` lit les fichiers RIS que vous avez exportés depuis l'interface WoS et les enregistre comme sources manuelles avec `method=database`.

Formats acceptés : RIS (Zotero, Mendeley, EndNote, Web of Science, Scopus), BibTeX, CSV avec ligne d'en-tête (noms de colonnes Scopus et WoS reconnus; sinon `title`, `year`, `doi`, `journal`, `authors`, `url`, `abstract`, `block`, `cited_by`) et listes de notices JSON (par exemple le `all_records.json` de l'exécution d'un collègue).

`--method` suit les catégories PRISMA 2020 pour les notices identifiées par d'autres méthodes : `database` (un export de base de données — rejoint la colonne des bases de données), `citation` (listes de références, articles citants), `website`, `organisation`, `expert` (la recommandation d'un collègue), `other`.

Vous pouvez aussi fournir des fichiers supplémentaires à un seul rapport sans les stocker : `report.py --records file.ris`.

6.3 Depuis Zotero, Mendeley et EndNote

Sortie : chaque exécution écrit du RIS (`all_records.ris`, `ris/` par bloc), du BibTeX (`all_records.bib`) et du CSL-JSON (`all_records.csl.json`); importez avec File → Import. Les résumés, DOI et URL sont conservés, et le nom du bloc arrive comme mot-clé (`block:NOV`), de sorte que les éléments importés sont déjà étiquetés.

Entrée : exportez une collection en RIS (Zotero : clic droit → Export Collection → RIS; Mendeley : File → Export → RIS; EndNoteX : File → Export → RefMan RIS) et ingérez-la comme ci-dessus. Il n'y a pas de connexion en direct à l'API Zotero (feuille de route).

7. Rapports

7.1 Une exécution

```
python report.py lit/runs/20260828T095041
python report.py --latest --level full --format html pdf
```

7.2 Le répertoire de recherche entier

```
python report.py --project
python report.py --project --outdir lit_topomat --level intermediate --format md html
```

Les rapports vont dans `lit/reports/<stamp>-<level>/`. Le rapport de projet ajoute un tableau des **Sources** (chaque exécution et source manuelle, sa date, sa méthode, ses notices et « nouvelles ici » — les notices uniques qu'aucune source antérieure n'avait trouvées), une **Chronologie** (nombres par bloc au fil des exécutions; quand les notices sont entrées dans le projet), un flux PRISMA avec les deux colonnes d'identification et, quand `journals.py` a été lancé, les indicateurs des revues.

7.3 Niveaux

Niveau	Sections
<code>simple</code>	métadonnées; sources; stratégie de recherche avec la chaîne exacte par backend; résumé des résultats; chronologie; flux PRISMA 2020 + liste PRISMA-S; 10 meilleures notices par bloc; suggestions
<code>intermediate</code>	+ chaque notice unique; recouvrement des sources (« trouvé ici seulement »); distributions par année / revue / auteur; indicateurs des revues; revues filtrées; erreurs; statistiques de libre accès; historique des nombres

Niveau	Sections
full	+ chaque notice avec résumé complet, liste d’auteurs et les sources qui l’ont trouvée ; listes brutes par source avant dédoublonnage ; les notices filtrées ; configuration des backends ; fichiers project.json et source.json ; le journal de l’exécution ; environnement

Tailles, d’après l’exemple fourni (quatre blocs, trois bases de données CC0, 1 226 notices uniques) : 6, 68 et 427 pages de PDF.

7.4 Formats

md (Markdown ; rendu sur GitHub), html (autonome, clair/sombre, imprimable, diagramme SVG), tex (LaTeX avec diagramme TikZ), pdf, txt (texte brut, diagramme ASCII). Le PDF est compilé à partir du LaTeX avec xelatex, lualatex ou pdflatex si l’un d’eux est installé, sinon avec pandoc, sinon par un moteur intégré qui met en page la version texte — l’option n’échoue jamais.

7.5 Filtres

Option	Effet
--since DATE, --until DATE	garder les membres (exécutions / sources manuelles) recherchés dans la fenêtre
--latest	le membre le plus récent seulement (projet) ; l’exécution la plus récente (mode simple)
--diff	ne garder que les notices <i>vues pour la première fois</i> dans la fenêtre — « ce que les recherches depuis DATE ont apporté »
--year-from Y, --year-to Y	année de publication
--backends a b	bases de données / sources à inclure (les sources manuelles sont manual:<name>)
--blocks A CD	blocs à inclure
--sources auto\ manual\ all	types de membres
--records FILE...	RIS/BibTeX/CSV/JSON supplémentaires comme source manuelle transitoire
--metric NAME --min-metric X	garder les notices dont l’indicateur de revue est au moins X (voir §8)
--min-citations N	seuil de citations
--oa-only	seulement les notices avec une copie légale en libre accès (nécessite les données de --pdfs ou project.py oa)
--top N, --sort cited\ year\ metric	taille et ordre du tableau
--basename, --out	radical du nom de fichier et répertoire de sortie

Les filtres sont listés dans le tableau des métadonnées du rapport et dans l’item 9 de PRISMA-S, pour qu’un rapport filtré ne soit jamais pris pour la recherche entière.

7.6 PRISMA

Le rapport comporte un diagramme de flux PRISMA 2020 (SVG en HTML, TikZ en LaTeX/PDF, ASCII en Markdown et texte) et une liste de contrôle PRISMA-S pour la déclaration de la recherche. L’outil remplit ce qu’il peut connaître : notices identifiées par base de données (sommées sur les exécutions en mode projet), notices identifiées par d’autres méthodes (sources manuelles par méthode), notices récupérées, retirées par automatisations

(le filtre de revues), doublons retirés, notices restant à trier. Il est explicite sur le fait qu'*identifiées* (ce que chaque base de données déclare) et *récupérées* (ce qui a été téléchargé dans la limite de `--limit`) diffèrent.

Les étapes que seul un humain peut connaître sont lues dans `prisma.json` (exécution unique) ou `screening.json` (répertoire de recherche); un modèle avec des valeurs `null` est écrit au premier rapport. Remplissez les entiers au fil du tri et relancez le rapport :

```
{"records_screened": 2216, "records_excluded": 2100,
 "reports_sought": 116, "reports_not_retrieved": 4, "reports_assessed": 112,
 "excluded_reasons": {"not topological": 60, "no experiment": 30},
 "other_sought": 12, "other_not_retrieved": 0, "other_assessed": 12,
 "other_excluded_reasons": {"duplicate of included": 3},
 "studies_included": 22, "reports_included": 31,
 "citation_searching": "reference lists of the 22 included studies",
 "prior_work": "none", "peer_review": "search strategy reviewed by the librarian"}
```

7.7 Suggestions

Fondées sur des règles, à la fin de chaque rapport : appels de backend en échec, blocs à des milliers de résultats, blocs de taille « nouveauté » (lire chaque résultat), plafond `--limit` atteint, une base de données à forte part de revues filtrées, aucun backend de qualité citation, recherche de libre accès non lancée, étapes PRISMA non remplies, pas d'indicateurs des revues, dérive des nombres entre exécutions et — en mode projet — l'absence de toute source manuelle.

7.8 Langues

```
python report.py --latest --lang pt-BR
python report.py --project --lang de --format pdf
python librarian.py --report-lang fr # the report written at the end of a run
```

`--lang` (`report.py`) et `--report-lang` (`librarian.py`) acceptent `en` (par défaut), `pt-BR`, `es`, `de` ou `fr`; un répertoire de recherche peut fixer sa propre valeur par défaut avec `"defaults": {"lang": "es"}` dans `project.json`, et une option explicite l'emporte. Seul le texte propre du rapport change — titres, en-têtes de tableau, les étapes PRISMA 2020 et le diagramme de flux dans chaque format, la liste PRISMA-S, les paragraphes explicatifs et les suggestions — avec le séparateur de milliers de la langue. Ce que l'outil a trouvé ou reçu est reproduit exactement tel quel, quelle que soit la langue : titres, résumés, auteurs et revues des notices, vos noms et notes de blocs, les chaînes de requête exactes, les noms des backends, les options citées dans le texte, les noms de fichiers, les sorties JSON et le journal de l'exécution incorporé. La sortie console, `run.log` et les journaux d'audit sont toujours en anglais, de sorte que les exécutions faites dans différentes langues restent consultables ensemble.

8. Indicateurs des revues

```
python journals.py fetch # every journal seen in the directory
python journals.py fetch --providers openalex --refresh
python journals.py import-scimago scimagojr_2024.csv --year 2024 [--all]
python journals.py import-jcr JCR_JournalResults_*.csv # Journal Citation Reports downloads
python journals.py import-csv other.csv --provider my_metric --year 2023 --name-col Journal --
value-col Value [--issn-col ISSN] [--delimiter ";"] # any name/value table; ISSN
python journals.py list --missing jcr_if # journals still to look up by hand
python journals.py show --metric scopus_citescore
```

Magasin : `lit/journals/metrics.json`, une entrée par revue indexée par ISSN (sinon par nom normalisé), valeurs conservées **par année et jamais écrasées** — récupérez à nouveau l'an prochain et le rapport montre la série.

Fournisseur	Clé	Indicateurs	Historique
openalex	aucune	openalex_2yr (citation moyenne sur 2 ans, un chiffre de type facteur d'impact), openalex_h, œuvres/citations par année	instantané sous l'année de récupération
scopus	SCOPUS_API_KEY	scopus_citescore, sjr, snip	historique complet par année
scimago	aucune ; téléchargez le CSV de l'année sur scimagojr.com	sjr, scimago_h, quartile	un fichier par année
jcr	licence	jcr_if	import uniquement

Le Journal Impact Factor (Clarivate JCR) est propriétaire : il n'y a pas d'API gratuite et l'outil ne le scrapera pas. Les utilisateurs sous licence téléchargent des CSV depuis la page *Browse journals* du JCR (600 lignes par téléchargement ; découpez par catégorie, puis par quartile) et les importent avec `journals.py import-jcr FILE...` — les colonnes et l'année du JIF sont détectées. `journals.py list --missing jcr_if` affiche les revues de votre répertoire encore sans valeur, c'est-à-dire la liste à consulter. Le protocole complet est dans `docs/JCR_IMPORT.md`. Pour un indicateur couvrant toutes les revues, le CSV SCImago (~30 000 revues, un téléchargement) est la voie pratique ; `--all` importe le fichier entier, par défaut seules les revues vues dans vos notices sont importées.

Dans les rapports : une colonne d'indicateur dans les tableaux de notices, « revues de cet ensemble par indicateur », un tableau d'évolution pour les revues ayant deux années ou plus enregistrées, et le filtre `--min-metric`. `--metric` choisit lequel (par défaut `openalex_2yr`, ou `defaults.metric` dans `project.json`).

9. Web of Science

La grammaire complète `TS=/NEAR` est dans l'Expanded API, rarement sous licence ; le niveau gratuit Starter rejette les booléens complexes. Web of Science est donc un travail manuel, rendu petit :

```
python wos_manual.py prep      # query files + CHECKLIST.md in WoS grammar
python wos_manual.py walk     # copies each query to the clipboard in turn
python wos_manual.py ingest   # RIS exports -> records, registered as manual sources
python wos_manual.py status
python wos_manual.py prep --queries other.json # a different query file (default ./queries.json)
```

La liste de contrôle encode les réglages de l'interface qui cassent silencieusement les requêtes (Core Collection, recherche Advanced, éditions, forme balisée contre forme nue).

10. Journaux et audit

Chaque script écrit `<outdir>/logs/<script>_<stamp>_<pid>.log` avec l'invocation exacte, les versions de l'outil et de Python, le répertoire de recherche, chaque avertissement et erreur, et le résultat. La sortie console est réduite par défaut ; `--verbose` montre tout, `--quiet` seulement les avertissements et erreurs ; `--log-dir` déplace les journaux. Les exécutions conservent en plus `run.log` (la transcription de la console) dans le répertoire de l'exécution.

11. Flux de travail

Une vérification de nouveauté (un après-midi). Écrivez 1–3 blocs de requête croisée ; `--counts-only` ; resserrez tout ce qui est dans les milliers ; exécution complète avec `--pdfs` ; lisez les Suggestions ; lisez chaque

résultat des petits blocs à la main ; notez ce que vous avez trié dans `prisma.json` ; relancez `report.py` ; importez le RIS dans Zotero.

Une recherche systématique sur un projet (des mois). `project.py init`. Relancez `librarian.py` à intervalles avec le même `queries.json`. Ingérez les sessions Web of Science et les exports des collègues. `report.py --project` pour la vue d'ensemble ; `--project --since <dernier rapport> --diff` pour ce qui est nouveau ; `journals.py fetch` chaque année. Remplissez `screening.json` au fil de l'eau ; le diagramme PRISMA se complète de lui-même, prêt pour le matériel supplémentaire.

Un laboratoire. Un répertoire de recherche par projet (`--outdir`) ; chacun a son propre index, son tri et ses rapports. Les dossiers de dépôt permettent aux collaborateurs de déposer des exports sans apprendre l'outil. Il n'y a délibérément pas de fusion entre projets : questions différentes, blocs différents.

Un exemple complet. `docs/WALKTHROUGH.md` (en anglais) mène un projet réel de `queries.json` à un diagramme PRISMA achevé, chaque commande incluse.

Avec un agent d'IA. Pointez-le vers `AGENTS.md` ; demandez-lui de rédiger `queries.json` à partir de votre question de recherche, de lancer les balayages et de parcourir le rapport avec vous. Le fichier de requête structuré, les archives JSON et le rapport ont été conçus pour être écrits et audités par un agent.

12. Fonctionnalités et limitations

Fonctionnalités : une requête structurée traduite en huit grammaires natives ; bases de données en configuration JSON (`--init-backends`) ; exécutions archivées et citables avec chaînes de requête exactes et historique des nombres ; points de reprise et Ctrl-C sûr ; un filtre de revues avec justificatifs ; cinq backends sans clé ; NASA ADS et INSPIRE pour la physique ; liens légaux vers des PDF en libre accès via Unpaywall ; rapports à trois niveaux en cinq formats avec PRISMA 2020 et PRISMA-S ; répertoires de recherche avec sources manuelles, provenance, chronologie et rapports différentiels ; indicateurs des revues avec série par année ; journaux d'audit ; une suite de tests hors ligne (301 vérifications) et CI.

Limitations, toutes par conception ou par le monde :

- Les nombres ne sont pas comparables entre bases de données ; les opérateurs de proximité sont abandonnés. Découvrez ici ; citez une base de données dans l'article.
- Les résultats Scopus exigent un droit institutionnel (réseau/VPN). L'API de Web of Science est rarement sous licence ; utilisez la voie manuelle.
- arXiv reçoit au plus deux groupes par bloc.
- `--limit` plafonne les notices par bloc et backend (les plus citées d'abord) ; les grands blocs sont une tranche, pas l'ensemble complet. Augmentez-le quand vous avez besoin d'exhaustivité.
- OpenAlex indexe des dépôts non curatés (~15 % de ses notices) ; filtrés par défaut, conservés dans `junk.json`.
- Pas de téléchargement de PDF (liens Unpaywall seulement), pas de boule de neige, pas de graphe de citations, pas de connexion en direct à Zotero/Mendeley (feuille de route) ; BibTeX et CSL-JSON sont écrits, pas relus depuis une bibliothèque Zotero.
- Indicateurs des revues : les valeurs OpenAlex sont des instantanés ; le facteur d'impact JCR est propriétaire et import uniquement ; l'appariement des revues par nom est imparfait quand une notice n'a pas d'ISSN.
- Le dédoublement se fait par DOI, sinon par les 90 premiers caractères du titre ; les paires prépublication/publication aux titres différents survivent comme deux notices.
- Google Scholar n'est pas et ne sera pas un backend (pas d'API ; le scraping viole ses conditions).

13. Tests

`python tests/test_librarian.py`

Hors ligne, bibliothèque standard uniquement, sans clés : les backends s'exécutent contre des réponses d'API enregistrées, le générateur de rapports contre des exécutions et des répertoires de recherche synthétiques, et la ligne de commande de chaque script est exercée de bout en bout. Le fichier est aussi un module `pytest` (`pytest tests/`). La CI lance `pyflakes` et la suite sur Linux, Windows et macOS sous Python 3.9 et 3.13.

14. Licence et conduite

Apache License 2.0. L'outil est construit pour que le respect des conditions d'utilisation de chaque base de données soit le chemin facile : API documentées uniquement, limites de débit respectées, une adresse de contact dans chaque requête, pas de scraping, pas de contournement de paywall.